

ChipAlign: Instruction Alignment in Large Language Models for Chip Design via Geodesic Interpolation

Chenhui Deng
NVIDIA
cdeng@nvidia.com

Yunsheng Bai
NVIDIA
yunshengb@nvidia.com

Haoxing Ren
NVIDIA
haoxingr@nvidia.com

Abstract—Recent advancements in large language models (LLMs) have expanded their application across various domains, including chip design, where domain-adapted chip models like ChipNeMo have emerged. However, these models often struggle with instruction alignment, a crucial capability for LLMs that involves following explicit human directives. This limitation impedes the practical application of chip LLMs, including serving as assistant chatbots for hardware design engineers. In this work, we introduce ChipAlign, a novel approach that utilizes a *training-free* model merging strategy, combining the strengths of a general instruction-aligned LLM with a chip-specific LLM. By considering the underlying manifold in the weight space, ChipAlign employs geodesic interpolation to effectively fuse the weights of input LLMs, producing a merged model that inherits strong instruction alignment and chip expertise from the respective instruction and chip LLMs. Our results demonstrate that ChipAlign significantly enhances instruction-following capabilities of existing chip LLMs, achieving up to a 26.6% improvement on the IFEval benchmark, while maintaining comparable expertise in the chip domain. This improvement in instruction alignment also translates to notable gains in instruction-involved QA tasks, delivering performance enhancements of 3.9% on the OpenROAD QA benchmark and 8.25% on production-level chip QA benchmarks, surpassing state-of-the-art baselines.

I. INTRODUCTION

Recent years have seen remarkable breakthroughs in large language models (LLMs) across a wide range of applications, including text summarization, machine translation, and conversational AI [30]. Models like GPT [1], Gemini [19], Claude [4], and LLaMA [8] series have transformed numerous industries by automating complex tasks, enhancing decision-making processes, and enabling creative problem-solving that traditionally required human expertise. Alongside this success, there is a growing trend of adapting LLMs to specific domains to meet specialized needs. Domain-adapted LLMs have been developed for fields such as healthcare [22], finance [24], law [5], and climate [20], where nuanced understanding and specialized knowledge are essential for enhancing model performance within these domains.

In the realm of chip design, ChipNeMo stands out as a prominent example of domain-adapted LLMs [14]. Built on the LLaMA2-70B foundation model, ChipNeMo leverages domain-adaptive pretraining (DAPT) and finetuning (DAFT) to imbue the model with specialized knowledge in circuits, bugs, and electronic design automation (EDA) scripts. Following ChipNeMo, several customized LLMs have recently been developed for EDA tasks. He et al. introduced AutoMage, an LLM finetuned on LLaMA2 to specialize in EDA tool utilization [23]. Subsequently, Sharma et al. [17] and Pu et al [16], developed LLMs tailored for QA and script generation tasks from OpenROAD.

However, these tailored LLMs typically exhibit diminished instruction alignment, a fundamental capability of general-purpose chat LLMs to follow human instructions, as demonstrated in Figure 2. This decline in instruction alignment limits the practical usability of chip LLMs, as they may struggle to respond effectively to user-directed commands, making them less versatile and reliable in real-world applications. For instance, when serving as single or multi-turn chatbots for hardware design engineers, it is critical that the chip

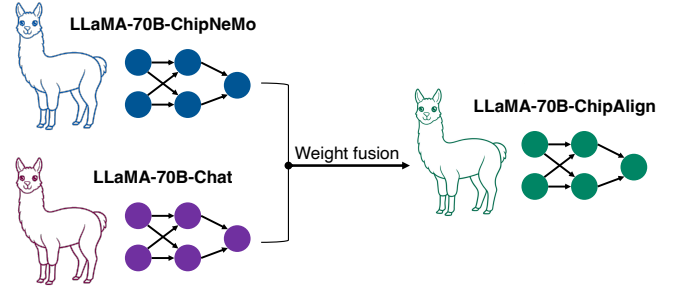


Figure 1: An illustration of model merging.

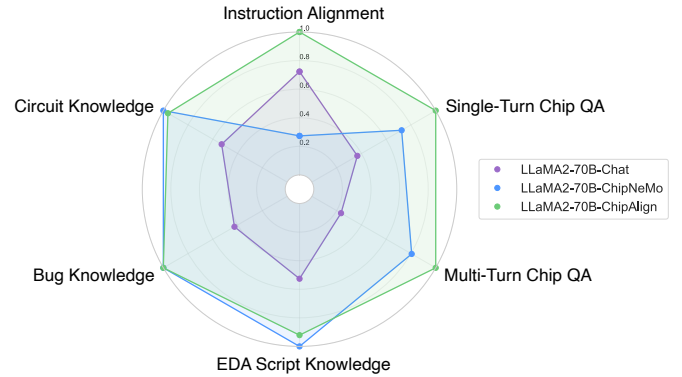


Figure 2: An overview of the capabilities of LLaMA2-70B variants: Chat, ChipNeMo, and ChipAlign on instruction alignment and chip domain benchmarks — Scores are normalized to $[0, 1]$ as per [12], with points closer to the outermost circle indicating better performance.

LLMs not only possess deep hardware knowledge but also strictly adhere to engineer instructions, such as “Please answer questions exclusively based on the provided context” or “Please provide a detailed and rigorous solution”. Unfortunately, prior chip LLMs often compromise this ability after going through DAPT or DAFT, leading to less satisfactory responses as indicated in Figures 5 and 6.

To overcome the aforementioned challenge, a viable solution is multi-task learning, training an LLM on both chip domain-specific and instruction-following data. This approach equips the model with dual capabilities, combining chip design expertise with a strong instruction-following capability. Nonetheless, a significant obstacle to multi-task learning is the proprietary nature of high-quality instruction datasets, such as those used even for open-sourced models like the LLaMA series [8], rendering it difficult to obtain a well-aligned model.

In this work, we address the instruction alignment challenge from an orthogonal perspective. Instead of training on instruction data, we leverage a *training-free* approach based on model merging, which fuses the weights of LLMs specialized in different tasks to produce a unified model that excels across them, as shown in Figures 1. To

this end, we introduce ChipAlign, which enhances the instruction alignment of a chip LLM by merging its weights with those of a well-aligned general instruction model that is publicly available (e.g., LLaMA2-Chat or LLaMA3-Instruct). More concretely, we treat the weights of both instruction and chip LLMs as two points on a Riemannian manifold, a curved geometric space that allows for measuring distances and angles [3]. To construct a merged model that embodies both strong instruction alignment and chip design expertise, it should ideally reside near both input models on this manifold. This requirement implies positioning the model along the geodesic—the shortest path between the two points on the manifold. Accordingly, ChipAlign employs geodesic interpolation to generate a new model that effectively combines the strengths of both input models.

To demonstrate the viability and flexibility of ChipAlign, we conduct extensive experiments across several benchmarks, including the instruction-following evaluation (IFEval) benchmark [32] and various chip domain benchmarks such as the open-source OpenROAD QA benchmark [16] and proprietary production-level chip QA benchmarks. We utilize LLMs ranging from 8B to 70B parameters. Our results indicate that ChipAlign significantly enhances the instruction-following capabilities of chip LLMs, thereby improving their performance on practical chip QA benchmarks that involve specific instructions. We summarize our main technical contributions as follows:

- To our knowledge, we are the first to apply a model merging approach to domain-adapted LLMs in chip design. This is achieved by fusing the weights of a chip LLM with those of an instruction-aligned LLM, without requiring additional training.
- By considering the geometric properties of the weight space in LLMs, ChipAlign utilizes geodesic interpolation to produce a merged model. This facilitates a smoother fusion of input LLM weights, leading to consistent enhancements in performance over previous model merging methods across different LLM backbones.
- Owing to the geometric-aware merging scheme in ChipAlign, the merged model inherits both strong instruction alignment and chip expertise from the input LLMs. Compared to the state-of-the-art baseline ChipNeMo, ChipAlign achieves a significant enhancement in instruction alignment, showing a 26.6% improvement on the IFEval benchmark while maintaining comparable domain knowledge.
- Thanks to the enhanced instruction alignment, ChipAlign outperforms state-of-the-art chip LLMs in instruction-involved QA tasks, achieving improvements of 3.9% on the OpenROAD QA benchmark and 8.25% on production-level chip QA benchmarks.

II. BACKGROUND

A. Domain-Adapted LLMs for Chip Design

In chip design, the need for domain-adapted LLMs has driven the development of several models tailored to hardware-related tasks. Liu et al. developed ChipNeMo starting with DAPT on 24 billion tokens drawn from chip design documents and code, using the LLaMA2-70B foundation model as a base. This pretraining phase employs the standard autoregressive language modeling objective to tailor the model to domain-specific data. Subsequently, the model underwent DAFT on approximately 57,000 samples, incorporating both domain-specific instructional data and open-source chat data from OASST [11]. Through DAPT and DAFT, ChipNeMo acquired specialized knowledge in the chip domain, leading to promising outcomes in various hardware design applications [14]. Later, He et al. developed AutoMage, an LLM finetuned on LLaMA2 for EDA tool usage [23], which led to the creation of ChatEDA, an autonomous agent customized for EDA design flow. More recently, Sharma et al. [17] and Pu et al. [16] have tailored LLMs for OpenROAD script generation and QA tasks,

covering a broad spectrum of queries related to command usage, VLSI flow, installation guides, and GUI usage.

B. Instruction Alignment in Chip Design LLMs

DAPT and DAFT often drastically change the weights of LLMs to emphasize domain knowledge, resulting in a loss of the instruction-following capabilities originally present in general-purpose LLMs [9]. However, instruction alignment is crucial for real-world applications, such as a chatbot assistant for chip designers. In such settings, designers may seek guidance on design methodologies, troubleshooting steps, or explanations of specific design concepts, often phrased as direct instructions. Additionally, they may instruct the chatbot to respond solely based on a given context, which ensures the answer is grounded in relevant and context-specific information, as shown in Figures 5 and 6. Hence, the ability to understand and respond appropriately to these instructions is vital for the practical usability of a chip LLM, making instruction alignment an essential feature.

A straightforward approach to enhance the instruction alignment of chip LLMs involves multi-task learning, which simultaneously trains a model on chip domain-specific data and instruction-following data to effectively integrate both sets of capabilities. However, access to high-quality instruction data is limited, as datasets used by advanced models like GPT-4 and the LLaMA series remain proprietary. While open-source instruction datasets are valuable [11], they often lack the scale and diversity needed to train models effectively for complex instruction-following tasks. Besides, even when data is available, the costs associated with finetuning on large-scale instruction datasets are prohibitively high, particularly for models with billions of parameters.

C. Instruction Alignment via Model Merging

In contrast to multi-task learning, model merging is a training-free technique that directly fuses the weights of specialized LLMs to incorporate multiple capabilities without requiring access to the original training data [27]. Recent work has demonstrated that with a properly designed weight fusion scheme, model merging can achieve performance comparable to multi-task learning, making it an efficient method to equip LLMs with multiple capabilities [28].

In literature, Model Soup represents a pioneering method in this direction, averaging the weights of different LLMs to create a single model that generalizes well across tasks [21]. Following this idea, Ilharco et al. proposed task arithmetic that averages the weight differences (i.e., task vectors) between input LLMs and their common base model; the resulting average is then added back to the base model to produce the merged model [10]. Building on task arithmetic, TIES [26] enhances the approach by sparsifying the task vectors and incorporating a sign consensus algorithm prior to weight fusion. DELLA [6] further advances TIES via adaptively pruning less important weights with specific hyperparameters. However, all of these methods neglect the underlying geometric properties of LLM weights, which can lead to merged models with suboptimal performance. In contrast, ChipAlign aims to merge a chip LLM with a well-instructed general LLM through geodesic interpolation. This geometric-aware technique allows us to smoothly blend the weights of the original LLMs along the shortest path on a Riemannian manifold. As a result, ChipAlign produces a merged model that effectively combines chip domain knowledge with instruction alignment, outperforming previous model merging techniques as detailed in Table 1. Notably, while ChipAlign has potential applications in other domains, this work primarily focuses on hardware-related QA tasks.

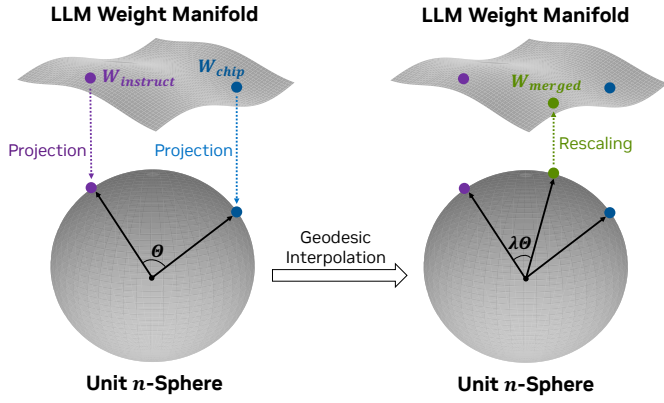


Figure 3: An overview of ChipAlign.

III. THE PROPOSED APPROACH

Problem formulation – Let M_{chip} denote a chip LLM and M_{instruct} denote a general-purpose instruction LLM. Let $\{W^{(l)} | l = 1, \dots, L\}$ denote the complete set of weights for an L -layer LLM, encompassing weights from the embedding layer, normalization layers, self-attention layers, and feed-forward layers. For each layer l , let $W_{\text{chip}}^{(l)} \in \mathbb{R}^{p \times q}$ and $W_{\text{instruct}}^{(l)} \in \mathbb{R}^{p \times q}$ represent the weight matrices of M_{chip} and M_{instruct} , respectively. Our goal is to develop a merging function f such that:

$$W_{\text{merge}}^{(l)} = f(W_{\text{chip}}^{(l)}, W_{\text{instruct}}^{(l)})$$

The resulting $W_{\text{merge}}^{(l)} \in \mathbb{R}^{p \times q}$ serves as the weights for the l -th layer in the merged model M_{merge} , which aims to combine the strengths of both M_{chip} and M_{instruct} . For brevity, we omit the layer index l in the following sections unless explicitly mentioned. Notably, our problem formulation implicitly assumes that the input models share the same architecture, meaning their respective weight matrices W_{chip} and W_{instruct} are conformable for merging. This assumption generally holds in practice; for example, ChipNeMo is trained based on LLaMA2-70B-Base, which has the same architecture as the instruction model LLaMA2-70B-Chat that is publicly available.

Figure 3 provides an overview of our proposed approach, ChipAlign. Next, we are going to first present the motivation for using geodesic interpolation in our merging method in Section III-A. Then, Section III-B introduces ChipAlign that computes geodesic interpolation along a unit n -sphere. Finally, we analyze the complexity of ChipAlign in Section III-C. It is worth noting that ChipAlign operates under the assumption that both chip and instruction LLMs are already available. Details on how to obtain these LLMs are provided in Section IV-A.

A. Model Merging via Geodesic Interpolation

Neural network weights can be viewed as points on a high-dimensional Riemannian manifold [2], [3]. This perspective opens the door for leveraging powerful geometric tools, such as geodesic interpolation, to transition smoothly between two sets of model weights. Geodesic interpolation follows the shortest path on the manifold between two points, providing a structured way to transition through the weight space without compromising on important model properties. Such interpolation techniques can be particularly useful for merging LLMs that specialize in different tasks. Given two points on the manifold corresponding to LLM weights optimized for distinct objectives, geodesic interpolation enables us to find a new point on the manifold that lies near both models, thereby inheriting strengths from each. This approach allows us to combine properties such as chip domain knowledge and instruction alignment into a single LLM.

However, a significant challenge arises due to the computational intractability of performing geodesic interpolation in high-dimensional manifolds. To overcome this, we project the model weights onto a specific type of Riemannian manifold, the unit n -sphere, where geodesic interpolation can be performed in linear time using a canonical form, as discussed in Section III-B. This method provides an efficient way to merge model capabilities while respecting the geometric properties of the high-dimensional weight space.

B. Geodesic Interpolation on Weight Manifold

Our approach ChipAlign adopts geodesic interpolation between the weights of a chip LLM and an instruction-aligned LLM. To this end, we first formally define the unit n -sphere in the following.

Definition III.1. The *unit n -sphere* S^n is the set of all points in $(n+1)$ -dimensional Euclidean space $\mathbb{R}^{n+1}$ that are at a unit distance from the origin:

$$S^n = \{w \in \mathbb{R}^{n+1} : \|w\| = 1\}$$

where $\|\cdot\|$ denotes the Euclidean norm.

To efficiently perform geodesic interpolation between the weights of these models, ChipAlign first projects the weight matrices W_{chip} and W_{instruct} onto a unit n -sphere as follows:

$$\begin{aligned} \text{Norm}_{\text{chip}} &= \|W_{\text{chip}}\|_F, & \text{Norm}_{\text{instruct}} &= \|W_{\text{instruct}}\|_F \\ \bar{W}_{\text{chip}} &= \frac{W_{\text{chip}}}{\text{Norm}_{\text{chip}}}, & \bar{W}_{\text{instruct}} &= \frac{W_{\text{instruct}}}{\text{Norm}_{\text{instruct}}} \end{aligned}$$

where $\|\cdot\|_F$ denotes the Frobenius norm. According to Definition III.1, $\bar{W}_{\text{chip}} \in \mathbb{R}^{p \times q}$ and $\bar{W}_{\text{instruct}} \in \mathbb{R}^{p \times q}$ now reside on a unit n -sphere ($n = p \times q - 1$), the geodesic between them is represented by the arc connecting the two points on the sphere. This allows us to perform geodesic interpolation by interpolating along this arc, which can be efficiently achieved using the following lemma:

Lemma III.2. Given $\bar{W}_{\text{chip}}$ and $\bar{W}_{\text{instruct}}$ lie on a unit n -sphere, the geodesic interpolation between them can be expressed as:

$$\bar{W}_{\text{merge}} = \frac{\sin(\lambda\Theta)}{\sin(\Theta)} \bar{W}_{\text{chip}} + \frac{\sin((1-\lambda)\Theta)}{\sin(\Theta)} \bar{W}_{\text{instruct}}$$

where $\Theta = \arccos(\langle \bar{W}_{\text{chip}}, \bar{W}_{\text{instruct}} \rangle)$ is the angle between $\bar{W}_{\text{chip}}$ and $\bar{W}_{\text{instruct}}$, and $\lambda \in [0, 1]$ determines the interpolation point along the geodesic, with $\lambda = 0$ corresponding to $\bar{W}_{\text{instruct}}$ and $\lambda = 1$ corresponding to $\bar{W}_{\text{chip}}$.

The proof for Lemma III.2 is available in [18]. Lemma III.2 allows us to generate a continuum of models between the two input models along the unit n -sphere, with λ determining the degree to which each model's weights are retained in the merged model.

Finally, to restore the magnitude of the original weight matrices, we rescale the interpolated weights back to the manifold by applying the Frobenius norms:

$$W_{\text{merge}} = \text{Norm}_{\text{chip}}^\lambda \cdot \text{Norm}_{\text{instruct}}^{1-\lambda} \cdot \bar{W}_{\text{merge}}$$

This process results in W_{merge} , the weight matrix for each layer in the merged model M_{merge} , effectively combining instruction alignment and chip design expertise in a single model.

Discussion – It is worth noting that an important advantage of ChipAlign is its ease of implementation, incorporating only a single hyperparameter λ . Our sensitivity analysis in Section IV-E reveals that $\lambda = 0.6$ leads to peak performance across different LLM backbones. This minimizes the burden of hyperparameter tuning,

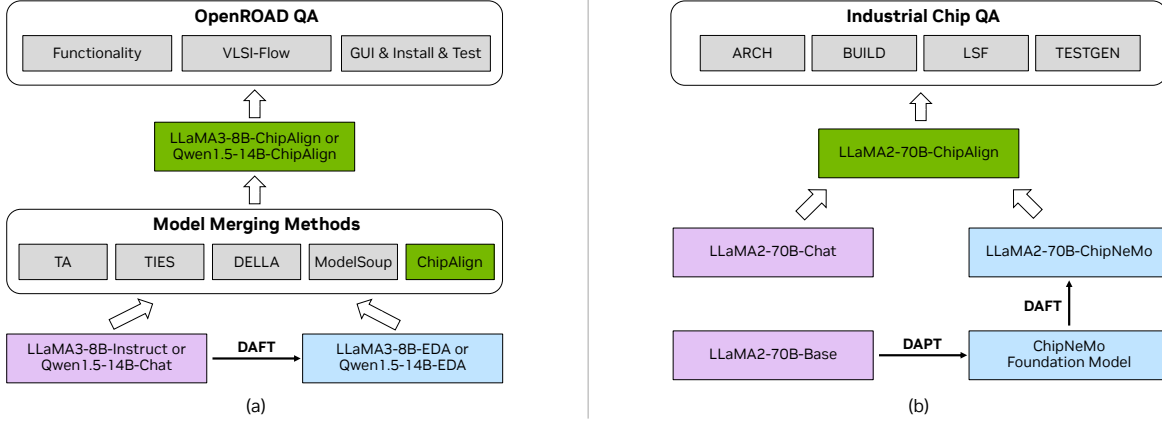


Figure 4: (a) An overview of developing merged models for the OpenROAD QA benchmark, featuring ChipAlign as the selected merging approach; (b) An overview of generating LLaMA2-70B-ChipAlign for industrial chip QA benchmarks.

making ChipAlign particularly easy to adopt for large-scale domain-adapted models. We further analyze the complexity of ChipAlign in Section III-C to highlight its efficiency advantages.

C. Complexity Analysis of ChipAlign

ChipAlign projects the weight matrices of input LLMs, each with n parameters, onto a unit n -sphere, requiring a single pass through the model weights in $O(n)$ time. It then performs geodesic interpolation along the arc between these weights and rescales the interpolated weights in $O(n)$ time. Consequently, the total time complexity for ChipAlign is $O(n)$. Similarly, its space complexity is $O(n)$, covering the storage of initial, projected, and final weights. This efficiency enables ChipAlign to handle billion-parameter scale LLMs with minimal computational overhead.

IV. EXPERIMENT

We have conducted a comprehensive evaluation of ChipAlign, comparing its performance against state-of-the-art (SoTA) baselines on the OpenROAD QA benchmark and industrial production-level chip QA benchmarks. To further evaluate ChipAlign's capabilities, we assess its instruction alignment on the IFEval benchmark and its domain knowledge on multiple-choice QA benchmarks covering EDA script generation, bug summarization, and circuit design. Finally, we conduct a sensitivity analysis on the hyperparameter λ in ChipAlign.

A. Experimental Setup

OpenROAD QA – As shown in Figure 4(a), we consider two well-aligned instruction LLMs that are publicly available: Qwen1.5-14B-Chat and LLaMA3-8B-Instruct. Following the methodology of Pu et al. [16] and Sharma et al. [17] to generate strong domain-adapted LLMs, we apply retrieval augmented DAFT to both models using around 2K context-query-answer training triplets from OpenROAD QA [29]. Specifically, we perform DAFT on each training QA pair along with its golden context, and adopt low-rank adaptation (LoRA) with a rank of 8 and an alpha of 16. We train the models over 20 epochs with a learning rate of 2×10^{-4} and a batch size of 1. Both models are trained on four nodes of a computing cluster, each node being equipped with eight A100 GPUs, each with 80GB of memory. This process results in the creation of Qwen1.5-14B-EDA and LLaMA3-8B-EDA. Once both the instruction and domain-adapted LLMs are available, we use ChipAlign with $\lambda = 0.6$ to fuse their weights, producing Qwen1.5-14B-ChipAlign and LLaMA3-8B-ChipAlign. Remarkably, the fusion process takes only 10 minutes on a CPU with 48 cores running at 2.5

GHz. Additionally, we compare ChipAlign with various popular model merging baselines such as task arithmetic (TA), TIES, DELLA, and Model Soup, adopting the recommended hyperparameters from their respective publications. Besides, we further compare ChipAlign against the general SoTA model GPT-4 Turbo and RAG-EDA, a Qwen1.5-14B-Chat based LLM highly customized for this benchmark [16]. To quantitatively assess model performance, we follow Pu et al. to adopt the ROUGE-L scoring method for comparing the generated LLM responses with the golden answers [13]. We have found that the ROUGE-L score is a more representative metric on this benchmark than either the BLEU [15] or UniEval [31] scores.

Industrial chip QA – As indicated by Figure 4(b), we select what is possibly the largest chip LLM, LLaMA2-70B-ChipNeMo, as our baseline. This model is developed by first undergoing DAPT on 24 billion tokens derived from chip design documents and code, using the LLaMA2-70B-Base model, followed by DAFT on 57K instruction pairs. More training details are available in [14]. Additionally, we choose LLaMA2-70B-Chat as the instruction model that is publicly available and has the same architecture as LLaMA2-70B-ChipNeMo. We set $\lambda = 0.6$ in ChipAlign for merging LLaMA2-70B-ChipNeMo and LLaMA2-70B-Chat to produce LLaMA2-70B-ChipAlign. Owing to the linear complexity of ChipAlign, the weight fusion process takes 43 minutes on a CPU equipped with 48 cores operating at 2.5 GHz. Besides, we utilize a GPT-4 based grader to assess the quality of each LLM response by comparing it with the golden answer. The grader assigns scores in $\{0, 25, 50, 75, 100\}$, where a higher score indicates better answer quality.

B. Evaluation on OpenROAD QA

We evaluate all models using 90 high-quality context-query-answer triplets from the OpenROAD QA benchmark, which all follow the same instruction shown in Figure 5. For each triplet, the context is derived either from the golden context corresponding to the given query or from the retrieved context via retrieval-augmented generation (RAG) using OpenROAD documentation. To establish a strong RAG pipeline, we employ the *bge-large-en-v1.5* for text embedding, *BM25* for lexical retrieval, and *bge-reranker-large* for re-ranking [25]. For the purpose of reproducibility, we set the temperature parameter to 0.0 for all models during response generation.

Table 1 showcases that ChipAlign surpasses all existing model merging methods in most cases, achieving a ROUGE-L score improvement of up to 6.4%. This empirical evidence confirms that our geometric-aware method produces a superior merged model compared to other

Input Prompt	Golden Answer	LLaMA3-8B-EDA	LLaMA3-8B-ChipAlign
Instruction You are the product consultant of an Electronic Design Automation (EDA) tool OpenROAD. Now given the user question and the related context, you are required to answer the question referring to the provided context. During answering the question, you have to follow these instructions: 1. Make your answer as rigorous as possible, do not fabricate the fact that does not mentioned in the provided context. 2. Your answer should be strongly related to the provided context, provide concrete solution for the answer and do not ignore the precondition in the query. Context: ## OpenROAD GUI ### Viewing Timing Report Click 'Timing' -> 'Options' to view and traverse specific timing paths. From Toolbar, click on the 'Timing' icon, View 'Timing Report' window added at the right side (RHS) of GUI as shown below. In 'Timing Report' Select 'Paths' -> 'Update', 'Paths' should be integer numbers. The number of timing paths should be displayed in the current window as follows: Select 'Setup' or 'Hold' tabs and view required arrival times and slack for each timing path segment. For each 'Setup' or 'Hold' path group, path details have a specific 'pin name, Time, Delay, Slew and Load' value with the clock to register, register to register and register to output data path. Question: A user has run the 'ibex' design in OpenROAD-flow-scripts from RTL-to-GDS. They now want to utilize the GUI to view setup and hold times of timing paths. How can they utilize the GUI to achieve this?	In the OpenROAD GUI, the user can view and traverse specific timing paths by navigating to 'Timing' -> 'Options'. In the 'Timing Report', select 'Paths' -> 'Update'. To view required arrival times and slack for each timing path segment, select either the 'Setup' or 'Hold' tabs. For each 'Setup' or 'Hold' path group, path details have a specific 'pin name, Time, Delay, Slew and Load' value with the clock to register, register to register and register to output data path. This information provides insight into the setup and hold times of the timing paths.	Inconcrete solution 😞 To view the setup and hold timing paths, open the OpenROAD GUI. In the GUI, navigate to 'Timing' -> 'Options'. Select 'Setup' or 'Hold' tabs and explore the arrival times and slack for each timing path segment. Remember that in the OpenROAD-flow-scripts, the 'ibex' design is used to demonstrate how to run the full RTL-to-GDS flow from the 'ibex' design. So, this is the design the user has run. For other designs, the approach is the same. Irrelevant to context 😞	In the OpenROAD GUI, click on the 'Timing' icon in the toolbar. This will open the 'View Timing Report' window on the right side of the GUI. In the 'Timing Report', select 'Paths' -> 'Update'. Choose the number of timing paths you want to view. Next, select either the 'Setup' or 'Hold' tabs. In these tabs, you will be able to view the required arrival times and slack for each timing path segment. Each path group will have specific information including pin name, time, delay, slew, and load value for the clock to register, register to register, and register to output data path. Accurate and concrete 😊

Figure 5: Comparison of model responses on the OpenROAD QA benchmark.

Table 1: ROUGE-L scores on the OpenROAD QA benchmark — * denotes the results sourced from [16].

Method	Golden Context				RAG Context			
	Functionality	VLSI Flow	GUI & Install & Test	All	Functionality	VLSI Flow	GUI & Install & Test	All
GPT-4 Turbo	0.280	0.338	0.367	0.315	0.250	0.274	0.331	0.276
RAG-EDA*	0.319	0.326	0.374	0.334	0.281	0.269	0.302	0.283
Qwen1.5-14B-Chat	0.276	0.318	0.354	0.305	0.244	0.264	0.301	0.263
Qwen1.5-14B-EDA	0.315	0.337	0.385	0.338	0.282	0.270	0.335	0.292
Qwen1.5-14B-TA	0.316	0.346	0.392	0.342	0.285	0.288	0.358	0.303
Qwen1.5-14B-TIES	0.300	0.352	0.368	0.329	0.255	0.279	0.338	0.281
Qwen1.5-14B-DELLA	0.295	0.343	0.382	0.328	0.254	0.278	0.304	0.272
Qwen1.5-14B-ModelSoup	0.317	0.359	0.389	0.345	0.295	0.278	0.358	0.306
Qwen1.5-14B-ChipAlign	0.354	0.366	0.403	0.369	0.305	0.294	0.354	0.314
LLaMA3-8B-Instruct	0.277	0.307	0.372	0.308	0.258	0.232	0.345	0.273
LLaMA3-8B-EDA	0.322	0.360	0.367	0.342	0.283	0.265	0.345	0.294
LLaMA3-8B-TA	0.284	0.362	0.372	0.325	0.267	0.292	0.344	0.292
LLaMA3-8B-TIES	0.286	0.332	0.376	0.319	0.256	0.281	0.347	0.285
LLaMA3-8B-DELLA	0.289	0.340	0.365	0.320	0.263	0.275	0.353	0.288
LLaMA3-8B-ModelSoup	0.333	0.370	0.429	0.365	0.288	0.286	0.368	0.307
LLaMA3-8B-ChipAlign	0.362	0.385	0.427	0.383	0.304	0.300	0.392	0.325

baselines. Moreover, the merged models generated by ChipAlign consistently outperform their corresponding EDA models across different LLM backbones, achieving performance improvements of up to 3.9%—a nontrivial margin for this benchmark. This improvement is attributed to ChipAlign’s efficacy in enhancing instruction alignment of domain-adapted models, resulting in responses that better align with the provided instructions, as evidenced in Figure 5 and Table 3.

C. Evaluation on Industrial Chip QA

We have extended our evaluation of ChipAlign to include industrial production-level chip QA benchmarks, consisting of 39 practical questions from hardware design engineers across domains such as hardware architecture (ARCH), build processes (BUILD), job scheduling (LSF), and verification (TESTGEN). In addition to a single-turn setting where each question is treated independently, we also explore a multi-turn setting to simulate real-world scenarios where engineers may pose follow-up questions based on previous interactions. As illustrated in Figure 6, each input prompt includes a question, relevant contexts obtained through RAG, and multiple instructions, necessitating strong instruction alignment in LLMs. We set the temperature parameter to 0.0 for all model responses.

Table 2 showcases that the merged model LLaMA2-70B-ChipAlign consistently outperforms both source models (LLaMA2-70B-Chat and LLaMA2-70B-ChipNeMo) by a margin of up to 8.25%. Besides, Figure 6 illustrates that both LLaMA2-70B-Chat and LLaMA2-70B-

ChipNeMo fail to follow the instructions, which mandate a model to answer questions using information from the relevant context. In contrast, LLaMA2-70B-ChipAlign strictly adheres to the provided instructions and generate correct answers supported by the context. This confirms ChipAlign’s efficacy with the largest existing chip LLMs, which is further analyzed in Section IV-D.

D. Evaluation on Instruction Alignment and Domain Knowledge

In addition to evaluating ChipAlign on both the OpenROAD QA and industrial chip QA benchmarks, which demand a joint capability of instruction alignment and domain knowledge, we further assess these two capabilities of ChipAlign separately.

Specifically, we assess instruction alignment using IFEval, a well-known benchmark designed to test LLMs’ ability to follow instructions. IFEval comprises 541 prompts featuring various general instructions, such as “List exactly 10 possible names ...” and “Think step-by-step and then give your answer ...”. Accuracy for each LLM is calculated based on the number of prompts or instructions that are either strictly (Strict Accuracy) or loosely (Loose Accuracy) adhered to.

Our results presented in Table 3 showcase that LLaMA3-8B-ChipAlign significantly outperforms LLaMA3-8B-EDA in instruction-following accuracy and matches the performance of LLaMA3-8B-Instruct. More importantly, LLaMA2-70B-ChipAlign not only improves the accuracy of LLaMA2-70B-ChipNeMo by 26.6% on average, but also considerably surpasses the source instruction model,

Input Prompt Instruction: - Please use the context below to answer questions. - Please adhere to the following format when the chunks do not provide enough information to answer the question: - I do not have enough information to answer this question. Context: Chunk[0]: Typical ZZZ tasks ... Chunk[1]: Commonly used options ... Chunk[2]: Options in ZZZ ... Question: How do I build a specific sandbox target using ZZZ?	LLaMA2-70B-Chat To build a specific sandbox target using ZZZ, you can use the <code>-target</code> option followed by the name of the target you want to build. For example, if you want to build a target called XXX, you can use the following command: <code>ZZZ -target XXX</code>. This will build the XXX target and all of its dependencies. If you want to build a specific version of the target, you can use the <code>-version</code> option followed by the version number. For example: <code>ZZZ -target XXX -version YYY</code>. This will build the XXX target with version YYY and all of its dependencies.	Not supported by context 😞 🗯️ Evaluation score: 25
Golden Answer To build a specific sandbox target using ZZZ, you can use the <code>-build</code> option followed by the name of the target. For example, if your target is named XXX, you would use the command <code>ZZZ -build XXX</code>. This will build the individual job with all its dependencies. If you want to build only the specific job without its dependencies, you can use the <code>-only</code> option instead, like <code>ZZZ -only XXX</code>.	LLaMA2-70B-ChipNeMo To build a specific sandbox target using ZZZ, you can use the following command: <code>ZZZ --only <target></code>. Where <code><target></code> is the name of the sandbox target to build. For example, if you want to build the XXX target, you can use the following command: <code>ZZZ --only XXX</code>. This will build only the specified target and its dependencies, without building the entire tree.	Not supported by context 😞 🗯️ Evaluation score: 50
LLaMA2-70B-ChipAlign To build a specific sandbox target using ZZZ, you can use the <code>-build</code> option followed by the name of the target. For example, to build the XXX target, you would use the following command: <code>ZZZ -build XXX</code>. In addition to the <code>-build</code> option, you can also use the <code>-only</code> option to build a specific job. For example, to build only the XXX job, you would use the following command: <code>ZZZ -only XXX</code>.	LLaMA2-70B-ChipAlign To build a specific sandbox target using ZZZ, you can use the <code>-build</code> option followed by the name of the target. For example, to build the XXX target, you would use the following command: <code>ZZZ -build XXX</code>. In addition to the <code>-build</code> option, you can also use the <code>-only</code> option to build a specific job. For example, to build only the XXX job, you would use the following command: <code>ZZZ -only XXX</code>.	Supported by context 😊 🗯️ Evaluation score: 100

Figure 6: An example from the BUILD category of the industrial chip QA benchmark — Due to space limitations, only the first few words of each context chunk are shown, omitting full context. Besides, we redact the build tool and target names using ZZZ and XXX, respectively.

Table 2: GPT4-aided evaluation scores on industrial production-level chip QA benchmarks.

Method	Single Turn					Multi Turn				
	ARCH	BUILD	LSF	TESTGEN	All	ARCH	BUILD	LSF	TESTGEN	All
LLaMA2-70B-Chat	40.00	70.50	54.25	66.75	57.00	10.00	41.00	10.50	50.00	30.25
LLaMA2-70B-ChipNeMo	45.00	81.75	60.50	87.50	66.75	40.00	81.75	52.00	33.25	54.50
LLaMA2-70B-ChipAlign	57.50	84.00	70.75	91.75	74.25	42.50	75.00	60.50	79.25	62.75

Table 3: Instruction-following accuracy (%) on IFEval.

Method	Prompt Level		Instruction Level	
	Strict	Loose	Strict	Loose
LLaMA3-8B-Instruct	67.5	75.4	76.1	82.6
LLaMA3-8B-EDA	60.0	64.6	69.6	74.1
LLaMA3-8B-ChipAlign	69.9	74.1	78.3	81.9
LLaMA2-70B-Chat	43.2	50.1	55.2	61.5
LLaMA2-70B-ChipNeMo	27.9	32.0	39.8	44.4
LLaMA2-70B-ChipAlign	54.3	61.0	65.0	70.3

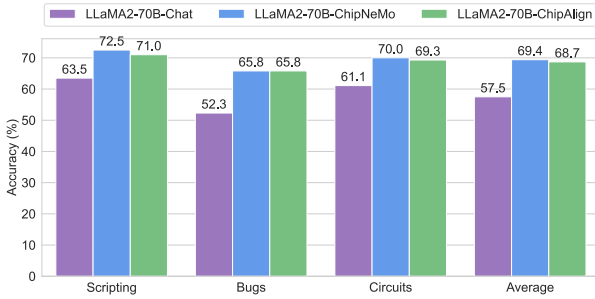


Figure 7: Results on multi-choice chip QA benchmark.

LLaMA2-70B-Chat. This enhanced performance can be attributed to the integration of the OASST instruction dataset [11] and the SteerLM alignment strategy [7] during the training of LLaMA2-70B-ChipNeMo, which imbued its weights with decent instructional knowledge complementary to that of LLaMA2-70B-Chat. Consequently, ChipAlign benefits from the combined instructional knowledge of LLaMA2-70B-ChipNeMo and LLaMA2-70B-Chat through the process of weight fusion. This fusion results in a merged model with stronger instruction alignment capabilities compared to both source models.

For evaluating pure chip domain knowledge, we utilize multi-choice chip QA benchmarks from [14] that contain no instructions. Figure 7 shows that ChipAlign performs on par with ChipNeMo across the domains of EDA scripts, bugs, and circuits, highlighting its ability to preserve domain knowledge after weight fusion. Furthermore,

the comparative performance of LLaMA2-70B-Chat, LLaMA2-70B-ChipNeMo, and LLaMA2-70B-ChipAlign is visualized in Figure 2, providing a comprehensive overview of their respective capabilities.

E. Sensitivity Analysis on λ

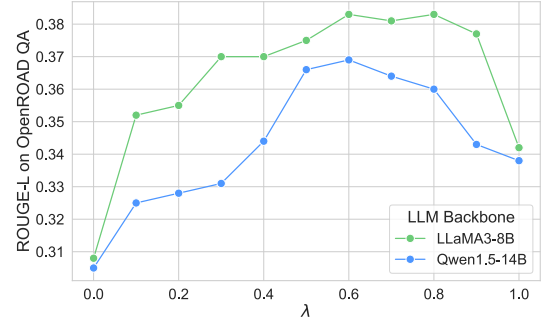


Figure 8: Sensitivity analysis on λ in ChipAlign.

As discussed in Section III, ChipAlign involves a single hyperparameter λ that determines how closely the merged model aligns with either the instruction model M_{instruct} or the chip model M_{chip} along the geodesic. We conduct a sensitivity analysis of λ using both LLaMA3-8B and Qwen1.5-14B backbones on the OpenROAD QA benchmark. Notably, $\lambda = 0$ and $\lambda = 1$ correspond to the M_{instruct} and M_{chip} , respectively. As depicted in Figure 8, model performance initially increases rapidly from M_{instruct} (i.e., the leftmost point), peaks at $\lambda = 0.6$, and subsequently declines towards the performance level of M_{chip} (i.e., the rightmost point). Hence, we recommend setting $\lambda = 0.6$ as the default value in ChipAlign for practical applications.

V. CONCLUSION

This work introduces ChipAlign, a geometric-aware model merging approach to enhance instruction alignment in chip LLMs. ChipAlign treats the weights of a chip LLM and a general instruction LLM as two points on a Riemannian manifold and employs geodesic interpolation to create the merged model. Our results demonstrate that this merged model significantly improves instruction alignment in chip LLMs, yielding superior performance across various chip QA benchmarks.

REFERENCES

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Shun-ichi Amari. Information geometry and manifolds of neural networks. In *From Statistical Physics to Statistical Inference and Back*, pages 113–138. Springer, 1994.
- [3] Shun-ichi Amari, Koji Kurata, and Hiroshi Nagaoka. Information geometry of boltzmann machines. *IEEE Transactions on neural networks*, 3(2):260–271, 1992.
- [4] Anthropic. The claude 3 model family: Opus, sonnet, haiku. *Technical Report*, 2024.
- [5] Jiayi Cui, Zongjian Li, Yang Yan, Bohua Chen, and Li Yuan. Chatlaw: Open-source legal large language model with integrated external knowledge bases. *arXiv preprint arXiv:2306.16092*, 2023.
- [6] Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. Della-merging: Reducing interference in model merging through magnitude-based sampling. *arXiv preprint arXiv:2406.11617*, 2024.
- [7] Yi Dong, Zhilin Wang, Makesh Narsimhan Sreedhar, Xianchao Wu, and Oleksii Kuchaiev. Steerlm: Attribute conditioned sft as an (user-steerable) alternative to rlhf. *arXiv preprint arXiv:2310.05344*, 2023.
- [8] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [9] Sreyan Ghosh, Chandra Kiran Reddy Evuru, Sonal Kumar, Deepali Aneja, Zeyu Jin, Ramani Duraiswami, Dinesh Manocha, et al. A closer look at the limitations of instruction tuning. *arXiv preprint arXiv:2402.05119*, 2024.
- [10] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.
- [11] Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi Rui Tam, Keith Stevens, Abdullah Barhoum, Duc Nguyen, Oliver Stanley, Richárd Nagyfi, et al. Openassistant conversations-democratizing large language model alignment. *Advances in Neural Information Processing Systems*, 36, 2024.
- [12] Yadong Li, Haoze Sun, Mingan Lin, Tianpeng Li, Guosheng Dong, Tao Zhang, Bowen Ding, Wei Song, Zhenglin Cheng, Yuqi Huo, et al. Baichuan-omni technical report. *arXiv preprint arXiv:2410.08565*, 2024.
- [13] Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81, 2004.
- [14] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Bayraktaroglu, et al. Chipnemo: Domain-adapted llms for chip design. *arXiv preprint arXiv:2311.00176*, 2023.
- [15] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- [16] Yuan Pu, Zhuolun He, Tairu Qiu, Haoyuan Wu, and Bei Yu. Customized retrieval augmented generation and benchmarking for eda tool documentation qa. *arXiv preprint arXiv:2407.15353*, 2024.
- [17] Utsav Sharma, Bing-Yue Wu, Sai Rahul Dhanvi Kankipati, Vidya A Chhabria, and Austin Rovinski. Openroad-assistant: An open-source large language model for physical design tasks. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, pages 1–7, 2024.
- [18] Ken Shoemake. Animating rotation with quaternion curves. In *Proceedings of the 12th annual conference on Computer graphics and interactive techniques*, pages 245–254, 1985.
- [19] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- [20] Saeid Ashraf Vaghefi, Dominik Stambach, Veruska Muccione, Julia Binger, Jingwei Ni, Mathias Kraus, Simon Allen, Chiara Colesanti-Senni, Tobias Wekhof, Tobias Schimanski, et al. Chatclimate: Grounding conversational ai in climate science. *Communications Earth & Environment*, 4(1):480, 2023.
- [21] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pages 23965–23998. PMLR, 2022.
- [22] Chaoyi Wu, Weixiong Lin, Xiaoman Zhang, Ya Zhang, Weidi Xie, and Yanfeng Wang. Pmc-llama: toward building open-source language models for medicine. *Journal of the American Medical Informatics Association*, page ocae045, 2024.
- [23] Haoyuan Wu, Zhuolun He, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. Chateda: A large language model powered autonomous agent for eda. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [24] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhjan Kambadur, David Rosenberg, and Gideon Mann. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.
- [25] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighof. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.
- [26] Prateek Yadav, Derek Tam, Leshem Choshen, Colin A Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36, 2024.
- [27] Enneng Yang, Li Shen, Guibing Guo, Xingwei Wang, Xiaochun Cao, Jie Zhang, and Dacheng Tao. Model merging in llms, mllms, and beyond: Methods, theories, applications and opportunities. *arXiv preprint arXiv:2408.07666*, 2024.
- [28] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Forty-first International Conference on Machine Learning*, 2024.
- [29] Tianjun Zhang, Shishir G Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E Gonzalez. Raft: Adapting language model to domain specific rag. *arXiv preprint arXiv:2403.10131*, 2024.
- [30] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.
- [31] Ming Zhong, Yang Liu, Da Yin, Yuning Mao, Yizhu Jiao, Pengfei Liu, Chenguang Zhu, Heng Ji, and Jiawei Han. Towards a unified multi-dimensional evaluator for text generation. *arXiv preprint arXiv:2210.07197*, 2022.
- [32] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.